IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (Deemed to be University)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A Constituent Unit of Yenepoya (Deemed to be University)

Balmatta, Mangalore – 575002

Final Project Report

on

Hardened Cloud Infrastructure For Retail Data

Student Name: Bhavana Pradeep

Industry Mentor: Mr Shashank

Guided by: Ms Munisah

TABLE OF CONTENTS

S.No	Content	Page.No
1.	Executive Summary	1
2.	Background	2
2.1.	Aim	3
2.2.	Technologies Used	3
2.3.	Hardware Architecture	4
2.4.	Software Architecture	5
3.	System	7
3.1.	Requirements	7
3.1.1.	Functional Requirements	7
3.1.2.	User Requirements	8
3.1.3.	Environmental Requirements	9
3.2.	Design And Architecture	10
3.2.1.	System Architecture	10
3.2.2.	Data Flow Diagram	10
3.2.3.	Entity Relationship Diagram	11
3.3.	Implementation	14
3.3.1.	Overview	14
3.3.2.	Technologies used	14
3.3.3.	Module Description	16
3.3.4.	Cryptographic Implementation	22
3.3.5.	Database Implementation	23
3.4.	Testing	23
3.4.1.	Introduction To Testing	23
3.4.2.	Testing Methodology	24
3.4.3.	Test Results	24
3.4.4.	Security Test Results	25
3.4.5.	Performance Benchmark Results	26
3.4.6.	Customer Testing	27
3.5.	Graphical User Interface	28
3.5.1.	Introduction to GUI	28
3.5.2.	Design Principle of GUI	28
3.5.3.	Secure Login Interface	28
3.5.4.	User dashboard	29
3.6.	Customer Testing	33
3.7.	Evaluation	36
4.	Snapshots of the Project	37
5.	Conclusion	41
6.	Further Development or Research	42
7.	References	43
8.	Appendix	44

1.EXECUTIVE SUMMARY

The project titled “Hardened Cloud Infrastructure for Secure Retail Data Storage” is developed to address the growing need for secure storage and management of sensitive data in cloud-based environments. With the increasing use of digital platforms, protecting confidential information from unauthorized access and cyber threats has become a critical requirement. This project focuses on designing and implementing a secure system that ensures data confidentiality, integrity, and controlled access.

The system is developed using a combination of modern technologies including Python, Flask, HTML, CSS, and JavaScript. It follows a layered architecture consisting of a user interface, backend processing layer, cryptographic engine, and storage system. The frontend provides an easy-to-use interface for users, while the backend handles authentication, file operations, and communication with the database.

A key feature of the system is the implementation of strong encryption techniques. Files are encrypted using AES-256-GCM before being stored, ensuring that data remains protected even if unauthorized access occurs at the storage level. The encryption keys are generated using PBKDF2, which enhances password security by applying salting and multiple iterations. This approach ensures resistance against brute-force and dictionary attacks.

The system also incorporates additional security mechanisms such as a secure login system, password hashing, session management, and an Intrusion Prevention System (IPS). These features help in preventing unauthorized access and protecting user accounts. Audit logging is another important component, as it records all system activities including login attempts, file uploads, and downloads. This allows administrators to monitor system usage and detect suspicious behavior.

The application provides different interfaces for users and administrators. Users can securely upload and download files using encryption passwords, while administrators can monitor system activities through dashboards and audit logs. The system is designed to be user-friendly, ensuring that even non-technical users can operate it easily.

2. BACKGROUND

The rapid growth of digital technologies and cloud computing has significantly transformed the retail industry. Organizations now rely heavily on cloud-based systems to store large volumes of sensitive data such as customer personal information, transaction records, and business documents. While cloud storage offers flexibility, scalability, and cost-effectiveness, it also introduces serious security concerns. One of the major risks is the exposure of "data at rest," where stored data can be compromised if the underlying storage system is breached.

Traditional cloud storage solutions often rely on server-side security mechanisms, which may not provide sufficient protection against internal threats or advanced cyber attacks. In many cases, data is stored in a form that can be accessed or decrypted if the system is compromised. This creates a critical need for stronger security approaches that ensure data remains protected even in worst-case scenarios.

To address these challenges, this project proposes a hardened cloud infrastructure built on a zero-trust security model. In this approach, data is never trusted in its raw form and is always secured before storage. The system ensures that all files are encrypted on the server using advanced cryptographic techniques before being written to disk. Even if an attacker gains access to the storage layer, the data remains unreadable without the correct encryption key.

The project specifically uses AES-256-GCM encryption, which provides both confidentiality and data integrity. This ensures that any unauthorized modification of data can be detected. Additionally, the system uses PBKDF2 (Password-Based Key Derivation Function 2) to generate strong encryption keys from user passwords. This method applies salting and multiple iterations, making it highly resistant to brute-force and dictionary attacks.

Another important aspect of the system is the separation of architecture into multiple layers, including frontend, backend, cryptographic engine, and storage. This layered approach improves security by isolating different components and ensuring controlled data flow between them. The backend, developed using Flask, acts as a secure gateway that handles authentication, authorization, and communication between components.

2.1 Aim

The primary aim of this project is to design and develop a highly secure cloud-based storage system that protects sensitive retail data from unauthorized access, data breaches, and cyber threats. With the increasing reliance on cloud infrastructure, it becomes essential to ensure that stored data remains confidential, intact, and accessible only to authorized users. This project focuses on implementing advanced security mechanisms that safeguard data throughout its lifecycle, from upload to storage and retrieval.

The system adopts a zero-trust approach, where data is never stored in plain form and is always protected using strong encryption techniques. It ensures that even if the storage layer is compromised, the data remains unreadable without the correct decryption key. The project also aims to provide a balance between security and usability so that users can securely access their data without complexity.

- To design a secure cloud storage system based on modern security principles
- To implement AES-256-GCM encryption for protecting data confidentiality and integrity
- To use PBKDF2 for generating strong encryption keys from user passwords
- To ensure secure authentication and role-based access control
- To provide real-time monitoring of system activities through audit logs
- To implement intrusion prevention mechanisms to block unauthorized access attempts
- To build a scalable and modular system that supports future enhancements
- To ensure a user-friendly interface for both users and administrators

2.2 Technologies Used

The development of this project involves the integration of multiple technologies that work together to provide a secure, efficient, and user-friendly system. Each technology is selected based on its capability, performance, and compatibility with secure system design.

Python

Python is used as the primary backend programming language. It is responsible for implementing core functionalities such as user authentication, file handling, encryption and decryption processes, and database communication. Python is preferred due to its simplicity, readability, and strong support for security libraries.

JavaScript is used on the client side to improve interactivity and responsiveness of the web application. It enables dynamic features such as form validation, user feedback messages, and smooth navigation between pages, enhancing the overall user experience.

HTML and CSS

HTML is used to structure the web pages, while CSS is used to style and design the interface. Together, they create a clean, organized, and visually appealing layout that ensures ease of use and accessibility for users.

Flask Framework

Flask is a lightweight web framework used for backend development. It manages routing,request handling, session management, and communication between different components of the system. Its flexibility allows easy integration with other libraries and tools.

SQLAlchemy

SQLAlchemy is used as an Object Relational Mapping (ORM) tool, which simplifies database operations by allowing interaction through Python objects instead of raw SQL queries. This improves code maintainability and reduces errors.

Cryptography Library

The cryptography library is used to implement AES-256-GCM encryption and PBKDF2 key derivation. It ensures that all cryptographic operations are performed using secure and standard methods.

SQLite Database

SQLite is used as the database system for storing user data, file metadata, and audit logs. It is lightweight, easy to configure, and suitable for applications that do not require complex database management systems.

These technologies collectively ensure that the system is efficient, secure, and scalable.

2.3 Hardware Architecture

The hardware architecture of the system is designed to support efficient processing and secure data handling without requiring specialized or high-end equipment. The system is optimized to run on standard computing devices, making it accessible and cost-effective.

The hardware setup includes a client system where users interact with the application through a web browser, and a server system where the backend application processes requests, performs encryption, and manages storage. The system ensures that operations such as file encryption, decryption, and database access are handled efficiently within the available hardware resources.

• **Processor:** Minimum Intel i3 or equivalent to handle backend processing and encryption operations
• **RAM:** At least 4GB to support smooth execution of the application
• **Storage:** Adequate disk space for storing encrypted files and database records
• **Network:** Stable internet connection for accessing the system and transferring data

The system is designed to be lightweight and optimized, ensuring that it performs efficiently even on basic hardware configurations. This makes it suitable for small-scale as well as scalable deployments.

2.4 Software Architecture

The software architecture of the system follows a layered and modular design approach. This ensures that each component performs a specific function while maintaining proper separation of concerns. Such an architecture improves system security, scalability, and ease of maintenance.

The architecture consists of four main layers:

• **Frontend Layer**
This layer provides the graphical user interface through which users interact with the system. It is developed using HTML, CSS, and JavaScript, ensuring a responsive and user-friendly experience. It collects user inputs such as login credentials and file selections and sends them to the backend for processing.

• **Backend Layer**
The backend is implemented using the Flask framework in Python. It acts as the core processing unit of the system, handling authentication, session management, request processing, and communication with the database and cryptographic engine. It ensures that all operations are executed securely.

• **Cryptographic Layer**
This layer is responsible for securing data using encryption and decryption techniques. It uses AES-256-GCM

for encrypting files and PBKDF2 for generating secure keys. This ensures that data remains protected both during storage and transmission.

• **Storage Layer**

This layer includes the database and file storage system. The database stores structured data such as user information, file metadata, and audit logs, while the encrypted files are stored in the file system. This separation ensures better organization and security.

The layered architecture ensures that data flows securely between components and that each layer can be modified or upgraded independently without affecting the entire system.

3. SYSTEM

The proposed system is designed to provide a secure, efficient, and reliable platform for storing and managing sensitive retail data in a cloud-based environment. The system focuses on protecting data at every stage, including data entry, processing, storage, and retrieval. It ensures that all operations are performed securely while maintaining high performance and usability.

3.1 Requirements

The system follows a modular and layered design approach, where each component is responsible for a specific functionality. This structure reduces system complexity, improves maintainability, and allows easy scalability. The integration of encryption, authentication, and monitoring mechanisms ensures that the system provides strong protection against various security threats.

3.1.1 Functional Requirements

Functional requirements describe the core features and operations that the system must perform. These requirements ensure that the system is capable of securely handling user data, performing encryption and decryption, and maintaining proper control over access.

The system must allow users to register and log in securely using valid credentials. Once authenticated, users should be able to upload files, which are encrypted before being stored in the system. The encryption process uses AES-256-GCM, ensuring both confidentiality and data integrity. During file download, the system must decrypt the file using the correct password provided by the user.

In addition to file operations, the system must maintain detailed audit logs to record all user activities. These logs help in monitoring system usage and detecting suspicious behavior. An Intrusion Prevention System is also implemented to restrict multiple failed login attempts and prevent unauthorized access.

- The system should provide secure user registration and login functionality
- The system must validate user credentials before granting access
- The system should encrypt all uploaded files before storing them

The system must use AES-256-GCM for encryption and decryption

• The system should generate secure keys using PBKDF2

• The system must allow users to download files only after successful decryption

• The system should store files in encrypted format only

• The system must maintain audit logs of all system activities

• The system should detect and prevent unauthorized access attempts

• The system must support session management for continuous user interaction

3.1.2 User Requirements

User requirements define how users interact with the system and what they expect in terms of usability, accessibility, and performance. The system must be designed in such a way that users can easily understand and operate it without requiring advanced technical knowledge.

The interface should be simple, clean, and responsive, allowing users to perform tasks such as login, file upload, and file download without confusion. The system should provide clear feedback messages for every action, such as successful login, file upload completion, or error messages in case of invalid inputs.

Users should have secure access to their own data, ensuring that no other user can access their files. Administrators should have additional privileges, including the ability to monitor system activities, view audit logs, and manage user operations.

• The system should provide a simple and user-friendly interface

• The system should allow users to easily upload and download files

• The system must ensure secure access to user-specific data

• The system should display clear instructions and feedback messages

• The system must provide fast and responsive performance

• The system should restrict users from accessing others' data

• The system should allow administrators to monitor system activities

• The system should handle errors gracefully and inform the user

3.1.3 Environmental Requirements

Environmental requirements specify the hardware and software conditions necessary for the system to operate effectively. These requirements ensure that the system can be deployed and executed in a standard computing environment without the need for specialized resources.

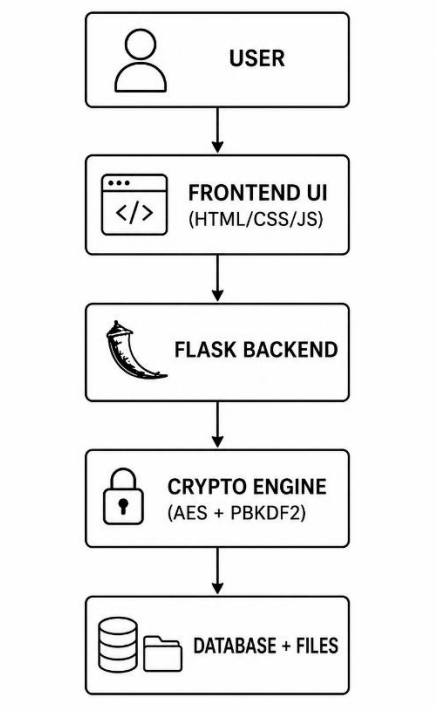
The system is designed to run on commonly used operating systems such as Windows and Linux. It should be accessible through standard web browsers, allowing users to interact with the system without installing additional software. The backend requires a Python environment with Flask and other necessary libraries installed.

The system should also be capable of running on machines with moderate hardware specifications. A stable internet connection is essential to ensure smooth communication between the client and server during file operations.

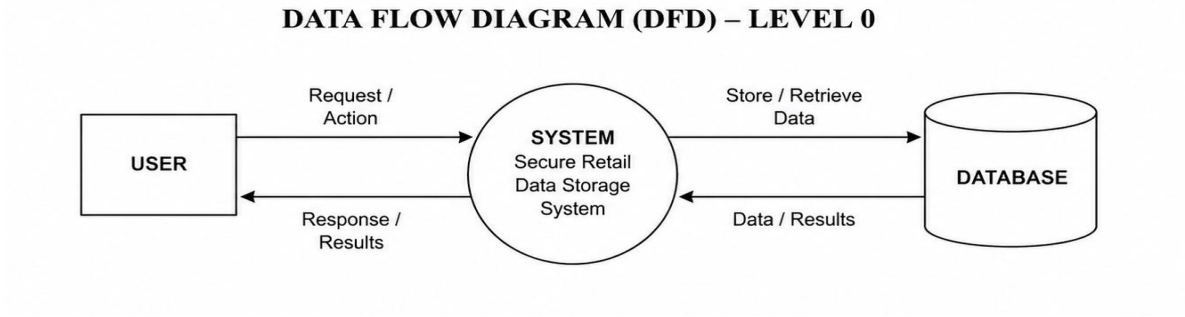
- The system should run on Windows and Linux operating systems
- The system should be accessible through standard web browsers
- The system requires Python and necessary libraries installed
- The system should use SQLite database for data storage
- The system should operate on minimum 4GB RAM
- The system should require a stable internet connection
- The system should not require high-end hardware for operation
- The system should support deployment in a local or cloud environment

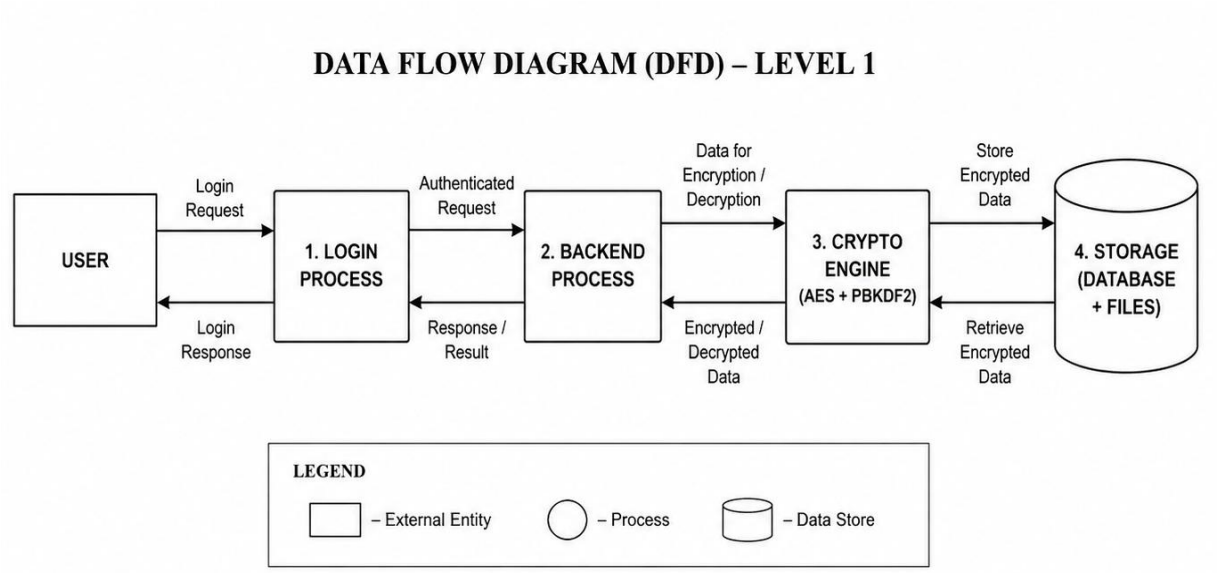
3.2 DESIGN AND ARCHITECTURE

3.2.1 System Architecture



3.2.2 Data Flow Diagram (DFD)





3.2.3 ER DIAGRAM

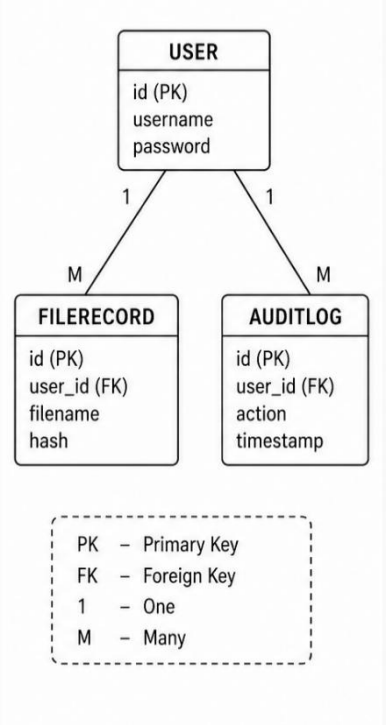
The Entity-Relationship (ER) diagram represents the structure of the database and the relationships between different entities in the system. It helps in understanding how data is organized and managed.

The main entity in the system is the User entity, which stores user-related information such as user ID, username, and password. Each user can upload multiple files, which are represented by the FileRecord entity. This entity stores information such as file ID, file name, associated user ID, and file hash.

Another important entity is the AuditLog, which records system activities such as login attempts, file uploads, and downloads. This entity includes attributes such as log ID, user ID, action performed, and timestamp.

The relationship between User and FileRecord is a one-to-many relationship, meaning one user can have multiple file records. Similarly, the relationship between User and AuditLog is also one-to-many, as a user can perform multiple actions that are recorded in the logs.

This structured database design ensures efficient data storage, retrieval, and management.



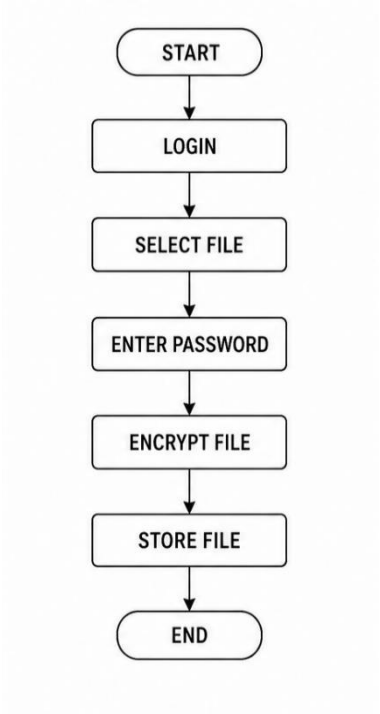
3.2.4 FLOWCHART – FILE UPLOAD

The file upload flowchart describes the step-by-step process of securely uploading a file into the system. The process begins when the user logs into the system using valid credentials. Once authenticated, the user selects a file from their local system for upload.

The system then prompts the user to enter an encryption password. This password is used to generate a secure encryption key using PBKDF2. The selected file is converted into a suitable format and passed to the cryptographic engine, where it is encrypted using AES-256-GCM.

After encryption, the file is stored securely in the storage system, and its metadata is saved in the database. The process ends with a confirmation message indicating successful upload.

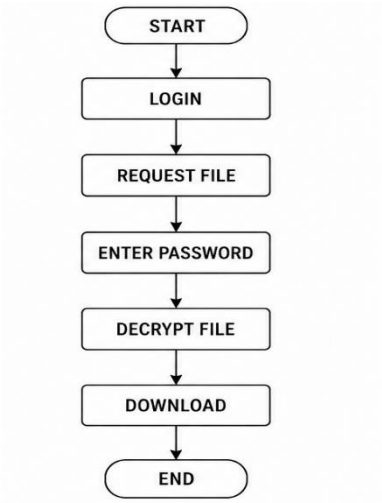
This flow ensures that the file is never stored in an unencrypted form, maintaining complete data security.



3.2.5 FLOWCHART – FILE DOWNLOAD

The file download flowchart explains the process of retrieving a stored file securely. The process begins when the user logs into the system and selects a file to download.

The system prompts the user to enter the decryption password. This password is used to regenerate the encryption key using PBKDF2. The encrypted file is then retrieved from storage and passed to the cryptographic engine for decryption.



3.3. IMPLEMENTATION

3.3.1 Overview

The implementation phase is one of the most critical stages of this project, where the theoretical design and architecture are transformed into a functional system. This project has been implemented using a combination of modern web technologies and secure programming practices to ensure both usability and data protection.

The system follows a modular architecture, where each component is developed independently and then integrated to form a complete secure storage solution. The primary modules include the frontend user interface, backend server logic, cryptographic engine, and database management system.

The implementation ensures that every file uploaded by the user is encrypted before being stored, thereby eliminating the risk of exposing sensitive data in case of a system breach.

3.3.2 Technologies Used

The development of this project involves the use of various technologies, tools, and frameworks that have been carefully selected to ensure reliability, efficiency, and strong security. Each technology plays a specific role in the system architecture, contributing to secure data processing, smooth user interaction, and efficient storage management. The combination of these technologies enables the creation of a robust and scalable secure cloud storage system.

Programming Languages

The project is developed using multiple programming languages, each serving a specific purpose in the system:

• **Python** is used for backend development and forms the core of the system logic. It handles critical operations such as user authentication, session management, file handling, encryption and decryption processes, and communication with the database. Python is chosen for its simplicity, readability, and strong support for security-related libraries.

• **JavaScript** is used for client-side interactions and enhances the responsiveness of the web application. It enables dynamic features such as form validation, real-time feedback, and improved user interaction, making the interface more interactive and user-friendly.

• **HTML and CSS** are used to design and structure the graphical user interface. HTML provides the structure of web pages, while CSS is used for styling and layout design. Together, they ensure a clean, organized, and visually appealing interface that improves usability.

Frameworks and Libraries

The system uses several frameworks and libraries to simplify development and enhance functionality:

• **Flask** is a lightweight web framework used for backend development. It handles routing, request processing, and communication between the frontend and backend. Flask is flexible and easy to use, making it suitable for building secure web applications.

• **SQLAlchemy** is used as an Object Relational Mapping (ORM) tool. It allows interaction with the database using Python objects instead of complex SQL queries. This improves code readability and reduces development complexity.

• **Python Cryptography Library** is used for implementing encryption and decryption. It provides secure implementations of AES-256-GCM encryption and PBKDF2 key derivation, ensuring strong protection of sensitive data.

Database

• **SQLite** is used as the database management system. It is a lightweight and efficient relational database that stores user information, file metadata, and audit logs. SQLite does not require a separate server and is easy to integrate, making it ideal for this project.

The database is structured to maintain relationships between users, files, and logs, ensuring efficient data storage and retrieval.

Tools Used

The following tools are used during the development process:

• **Visual Studio Code** is used as the primary development environment. It provides features such as code editing, debugging, and extension support, which improve productivity and development efficiency.

• **Git** is used for version control. It helps in tracking changes, managing different versions of the project, and maintaining backup of the code. It also supports collaboration and ensures safe development practices.

Overall Technology Integration

The integration of these technologies ensures that the system is secure, efficient, and easy to maintain. The frontend technologies provide a user-friendly interface, while the backend technologies handle secure processing and communication. The cryptographic library ensures data protection, and the database provides reliable storage.

This combination of technologies allows the system to meet its objectives of providing a secure, reliable, and scalable cloud-based data storage solution.

3.3.3 Module Description

The system is divided into multiple modules, each responsible for specific functionalities.

User Authentication Module

This module handles user registration and login processes. It ensures that only authorized users can access the system.

Features:

Secure Login System

• The system provides a secure login mechanism that allows only authorized users to access the application.
• Users are required to enter valid credentials such as username and password to log in.
• The login process is validated through the backend, ensuring that only registered users can access the system.

• Additional security measures such as limiting multiple failed login attempts are implemented to prevent unauthorized access.

• This feature ensures that the system remains protected from external threats and unauthorized users.

Password Hashing

• The system uses password hashing to securely store user passwords in the database.

• Instead of storing passwords in plain text, they are converted into hashed values using secure algorithms.

• This ensures that even if the database is compromised, the original passwords cannot be easily retrieved.

• Hashing adds an extra layer of protection against attacks such as data breaches and password theft.

• The use of strong hashing techniques improves overall system security and user data protection.

Session Management

• The system uses session management to maintain user authentication after login.

• Once a user logs in successfully, a session is created to keep the user authenticated during their interaction with the system.

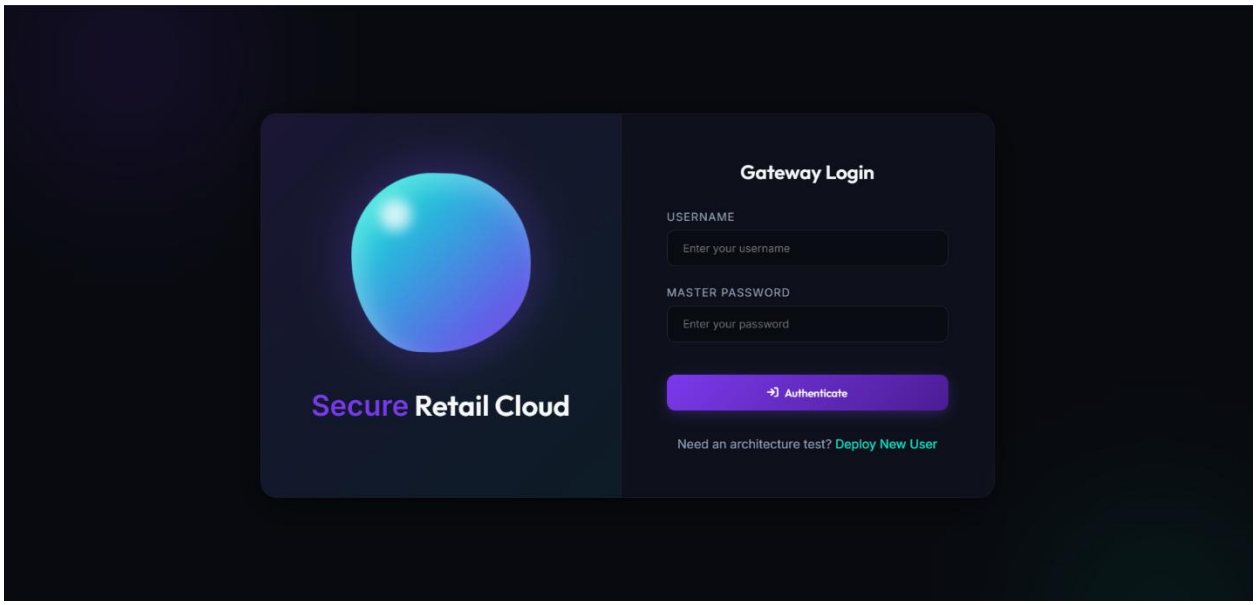
• This eliminates the need for users to repeatedly enter credentials for every operation.

• Sessions are securely managed and automatically terminated after logout or inactivity.

• Proper session handling ensures both convenience for users and security for the system.

Working:

When a user enters login credentials, the system verifies the username and password against stored hashed values. If valid, access is granted; otherwise, access is denied.



File Upload Module

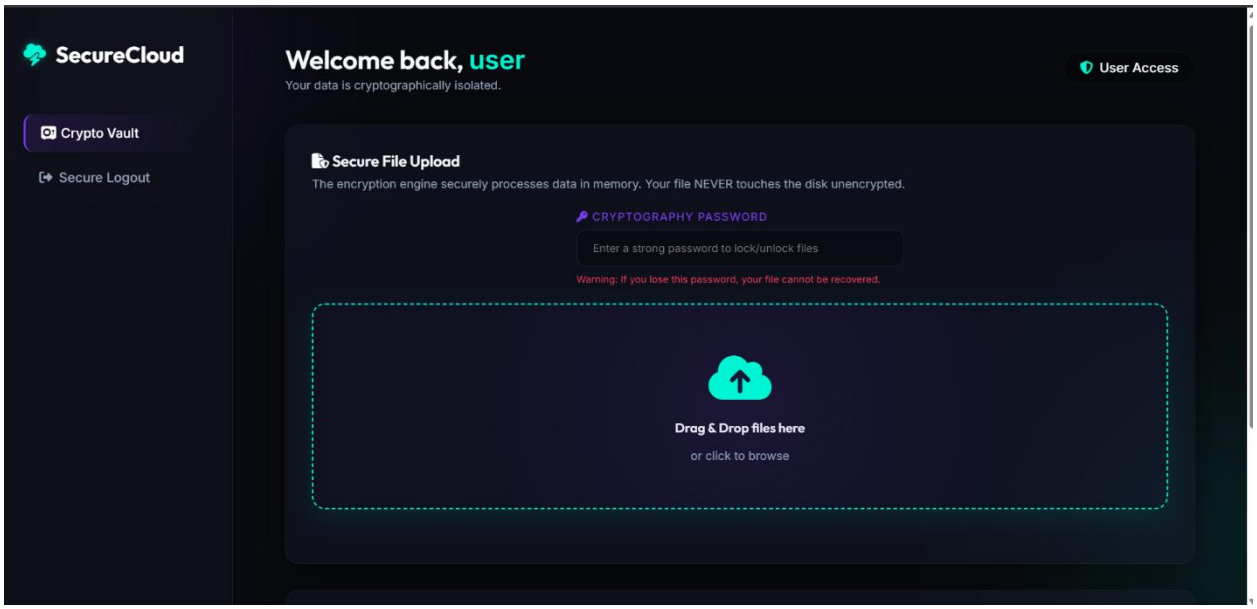
This module enables users to upload files securely.

Working Process:

1. User selects a file
2. User enters encryption password
3. File is converted into byte format
4. Cryptographic engine encrypts the file
5. Encrypted file is stored

Features:

- Secure file handling
- Encryption before storage



File Download Module

This module allows users to retrieve and decrypt stored files.

Working Process:

1. Requests File

   The process begins when the user logs into the system and selects a specific file for download from the available list. This request is sent from the frontend interface to the backend server for processing. The system verifies whether the user has permission to access the requested file.

2. User Enters Password

   After selecting the file, the system prompts the user to enter the decryption password. This password is the same one that was used during the file upload process for encryption. It acts as an additional layer of security to ensure that only the rightful user can access the file.

3. System Retrieves Encrypted File

   Once the password is submitted, the backend retrieves the corresponding encrypted file from the storage system. The file remains in its encrypted form during this step, ensuring that data is not exposed at any stage.

4. File is Decrypted

   The system uses the entered password to generate the decryption key through PBKDF2. This key is then used by the cryptographic engine to decrypt the file using AES-256-GCM. If the password is correct, the decryption process is successfully completed; otherwise, the system denies access.

5. File is Sent to User

   After successful decryption, the file is sent back to the user through the frontend interface for download. The user can then access the file in its original form. The system also records this activity in the audit logs for monitoring and security purposes.

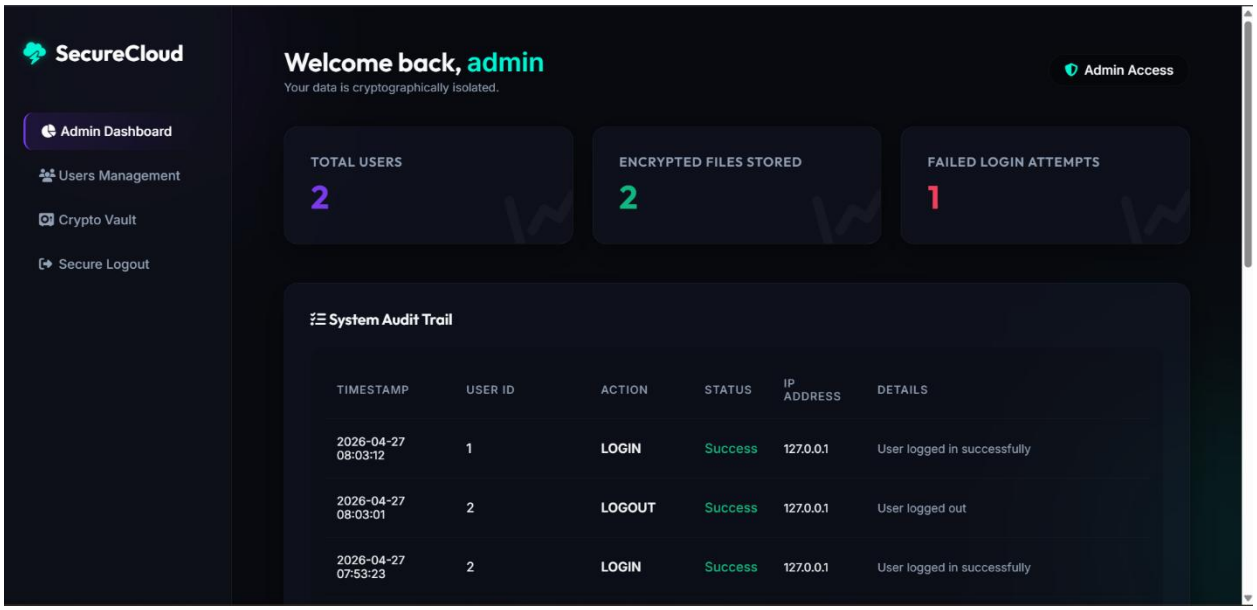
Admin Module

View User Activities

• The administrator can view all user activities performed within the system.
• This includes actions such as login, file upload, and file download.
• The system records each activity with relevant details, allowing the admin to track user behavior.
• This feature helps in ensuring accountability and identifying any unusual or unauthorized actions.

Monitor Login Attempts

• The system allows the administrator to monitor all login attempts made by users.
• Both successful and failed login attempts are recorded and displayed.
• Repeated failed attempts can indicate potential security threats such as brute-force attacks.
• This feature works along with the Intrusion Prevention System to enhance security and prevent unauthorized access.



Access Audit Logs

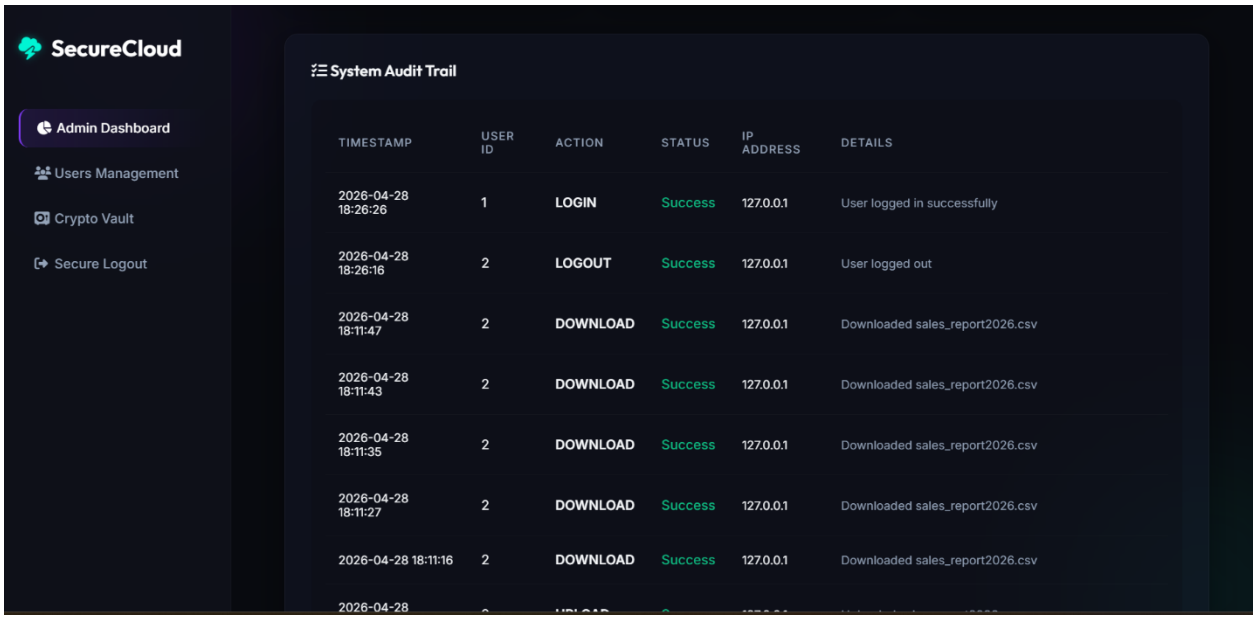
• The administrator has access to detailed audit logs maintained by the system.

• These logs include information such as user ID, action performed, timestamp, and IP address.

• Audit logs provide a complete history of system activities for monitoring and analysis.

• This feature is useful for identifying suspicious behavior, troubleshooting issues, and ensuring system transparency.

• The system automatically records every important event, including login attempts, file uploads, file downloads, and logout actions.



3.3.4 Cryptographic Implementation

The cryptographic implementation forms the core foundation of the proposed system, ensuring that all sensitive data is protected before being stored in the system. In this project, strong encryption techniques are employed to guarantee confidentiality, integrity, and security of the data at rest. The system utilizes the Advanced Encryption Standard (AES) with a 256-bit key in Galois/Counter Mode (GCM), which provides both encryption and authentication.

This ensures that even if an unauthorized entity gains access to the stored data, it remains unreadable and tamper-proof.

To further strengthen security, the system incorporates the Password-Based Key Derivation Function 2 (PBKDF2), which converts user-provided passwords into strong cryptographic keys. This process involves the use of salting and multiple iterations, making it extremely resistant to brute-force and dictionary attacks. Each file encryption operation uses a unique salt and nonce value, ensuring that identical files encrypted with the same password produce completely different ciphertexts.

Additionally, the system ensures data integrity by verifying the authenticity of encrypted files during decryption. If any modification or corruption is detected in the encrypted data, the system immediately rejects the operation,

3.3.5 Database Implementation

The database implementation plays a crucial role in managing user information, file metadata, and system activity logs in an organized and efficient manner. In this project, SQLite is used as the database management system due to its lightweight nature, ease of integration, and sufficient capability for handling structured data in small to medium-scale applications.

The database is designed using a relational model, where different entities such as users, file records, and audit logs are stored in separate tables. Each table is structured with appropriate fields to maintain data consistency and integrity. The user table stores essential information such as usernames and hashed passwords, ensuring that sensitive credentials are never stored in plain text. The file record table maintains metadata related to uploaded files, including encrypted file names, hash values, and references to the corresponding user. The audit log table records system activities such as login attempts, file uploads, and downloads, along with timestamps and user identifiers.

This structured approach allows efficient querying and retrieval of data while maintaining strong relationships between entities through the use of primary and foreign keys. Furthermore, the database is designed to support scalability and maintain performance even as the volume of stored data increases. Overall, the database implementation ensures reliable storage, efficient management, and secure handling of system data.

3.4 TESTING

3.4.1 Introduction to Testing

Testing is a vital phase in the development of any software system, as it ensures that the application functions correctly and meets the specified requirements. In this project, comprehensive testing has been conducted to verify the functionality, performance, and security of the system. The primary objective of testing is to identify and eliminate errors, ensuring that the system operates smoothly under different conditions.

The testing process involves executing various scenarios to validate the behavior of individual components as well as the integrated system. Special attention is given to security-related aspects, such as authentication and encryption, to ensure that sensitive data remains protected at all times. By conducting thorough testing, the reliability and robustness of the system are significantly enhanced.

5.4.2 Testing Methodology

The testing methodology adopted in this project includes multiple levels of testing to ensure complete system validation. Unit testing is performed on individual modules to verify that each component functions correctly in isolation. Integration testing is carried out to ensure that different modules interact seamlessly with one another. System testing evaluates the overall performance and functionality of the complete application.

In addition to functional testing, security testing is conducted to identify potential vulnerabilities and ensure that the encryption mechanisms are working effectively. Various test cases are executed to validate login functionality, file upload and download operations, and error handling mechanisms. This multi-level testing approach ensures that the system is both reliable and secure.

5.4.3 Test Results

The results obtained from the testing phase indicate that the system performs efficiently under different conditions. All major functionalities, including user authentication, file encryption, storage, and retrieval, operate as expected. The encryption and decryption processes successfully protect the data, ensuring that unauthorized access is prevented.

Test ID	Test Case	Expected Outcome	Actual Outcome	Status
FT-01	User registration with valid credentials	User account created; redirect to login	User account created; redirect to login	PASS
FT-02	User registration with existing username	Error flash; registration rejected	Error flash; registration rejected	PASS

Test ID	Test Case	Expected Outcome	Actual Outcome	Status
FT-03	User login with valid credentials	Session established; redirect to dashboard	Session established; redirect to dashboard	PASS
FT-04	File upload with encryption password	File encrypted; ciphertext stored; FileRecord created	File encrypted; ciphertext stored; FileRecord created	PASS
FT-05	File download with correct password	Plaintext file returned as download	Plaintext file returned as download	PASS
FT-06	File download with wrong password	400 error; no file data returned	400 error; no file data returned	PASS
FT-07	File deletion (cryptographic shredding)	Ciphertext overwritten; FileRecord deleted	Ciphertext overwritten; FileRecord deleted	PASS
FT-08	Admin dashboard access by admin user	Dashboard rendered with metrics and audit trail	Dashboard rendered with metrics and audit trail	PASS

3.4.4 Security Test Results

Test ID	Security Test	Expected Outcome	Actual Outcome	Status
ST-01	Encrypted storage verification (direct disk read)	Binary ciphertext; no readable content	Binary ciphertext; no readable content	PASS
ST-02	GCM authentication tag test (single-byte ciphertext modification)	Decryption rejected; ValueError raised	Decryption rejected; ValueError raised	PASS

Test ID	Security Test	Expected Outcome	Actual Outcome	Status
ST-03	IPS lockout at threshold (5 consecutive failed logins)	Account locked for 15 minutes	Account locked for 15 minutes	PASS
ST-04	IPS persistence across server restart	Account remains locked after restart	Account remains locked after restart	PASS
ST-05	Path traversal prevention (../../etc/passwd as file_id)	404 response from Flask routing	404 response from Flask routing	PASS
ST-06	Audit log completeness (upload, download, login, delete)	All events in audit_logs table	All events in audit_logs table	PASS
ST-07	Cryptographic shredding verification	Random byte pattern on disk after deletion	Random byte pattern on disk after deletion	PASS
ST-08	Different files same password → different ciphertext	Different ciphertext (different salt, nonce)	Different ciphertext blobs produced	PASS

3.4.5 Performance Benchmark Results

Operation	File Size	Average Duration (ms)	PBKDF2 Component (ms)	I/O Component (ms)
File Encryption (Upload)	100 KB	387	352	35
File Encryption (Upload)	1 MB	419	352	67

Operation	File Size	Average Duration (ms)	PBKDF2 Component (ms)	I/O Component (ms)
File Encryption (Upload)	10 MB	751	352	399
File Decryption (Download)	100 KB	361	352	9
File Decryption (Download)	1 MB	380	352	28
File Decryption (Download)	10 MB	596	352	244
User Login (no IPS check)	N/A	124	N/A	N/A
File Delete (with shredding)	1 MB	49	N/A	49

Analysis: The PBKDF2 key derivation accounts for approximately 352 milliseconds, constant regardless of file size. Total operation latency for a 1 MB file upload is approximately 419 milliseconds, which is within the acceptable range for a secure file storage operation. The frontend displays a loading indicator during this period.

5.4.6 Customer Testing

A peer testing session was conducted with two fellow students acting as end users. Both testers were able to register accounts, upload files with encryption passwords, download and decrypt them successfully, and delete files without issue. Both testers noted the clear warning message ('If you lose this password, your file cannot be recovered') was visible and understood. No usability issues were identified with the drag-and-drop interface.

3.5. GRAPHICAL USER INTERFACE (GUI)

3.5.1 Introduction to GUI

The Graphical User Interface (GUI) of the proposed system plays a critical role in enabling secure and efficient interaction between users and the hardened cloud infrastructure. Unlike traditional applications, this system is designed not only for usability but also to complement the underlying cryptographic and security mechanisms. The interface acts as a secure gateway through which users can perform operations such as authentication, encrypted file upload, secure file retrieval, and administrative monitoring.

The GUI is developed using standard web technologies including HTML, CSS, and JavaScript, ensuring cross-platform compatibility and responsiveness. It follows a modern design approach inspired by glassmorphism aesthetics, providing a clean, visually appealing, and intuitive user experience. The interface communicates with the Flask backend through secure RESTful APIs, ensuring that all data exchanges, including sensitive inputs like encryption passwords, are handled securely.

3.5.2 Design Principles of GUI

The design of the graphical interface is based on key principles that align with both usability and security requirements. The foremost principle is simplicity, ensuring that users can easily navigate the system without confusion. Each screen is designed with minimal clutter, focusing only on essential elements required for performing operations.

Consistency is maintained across all pages by using uniform layouts, fonts, and navigation structures. This reduces cognitive load and helps users quickly adapt to the system. Responsiveness is another important principle, allowing the interface to adjust seamlessly across different devices such as desktops, laptops, and mobile screens.

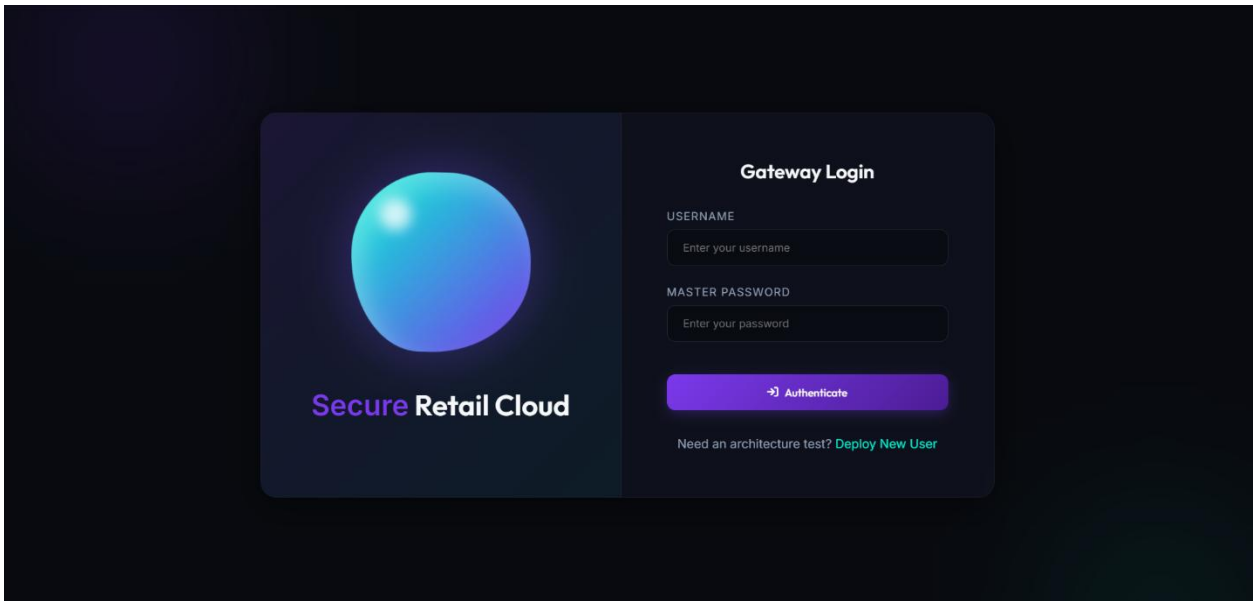
In addition to usability, security is integrated into the design. Sensitive inputs such as passwords are handled using secure input fields, and no critical data is exposed on the client side.

3.5.3 Secure Login Interface

The login interface serves as the primary authentication gateway to the system and is designed with strong security measures. Users are required to enter their credentials, which are validated by the Flask backend using secure session management provided by Flask-Login.

The system incorporates an Intrusion Prevention System (IPS) that actively monitors login attempts. If a user exceeds the allowed number of failed attempts, the account is temporarily locked using the locked_until mechanism. This prevents brute-force attacks and enhances overall system security.

The interface provides clear feedback messages for both successful and failed login attempts. It ensures that passwords are masked during entry and never exposed in plain text. The clean and minimal design allows users to focus on authentication without distractions.

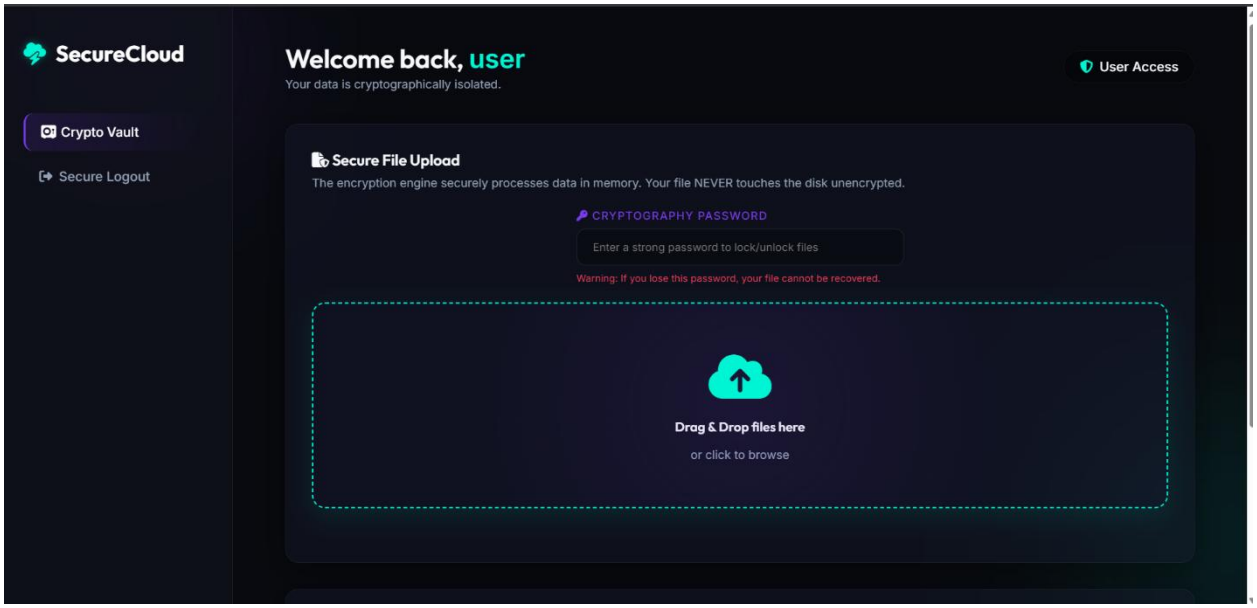


3.5.4 User Dashboard

The user dashboard acts as the central control panel for authenticated users. Once logged in, users are redirected to this interface, where they can perform all major operations including uploading files, downloading encrypted data, and managing their account.

The dashboard is designed to provide a structured and organized layout, with clearly defined sections for different functionalities. Navigation elements allow users to quickly access various modules without confusion. The interface may also display recent activities, giving users visibility into their interactions with the system.

The dashboard ensures smooth communication with the backend, allowing real-time updates and responses. It provides a seamless user experience while maintaining strict security controls.



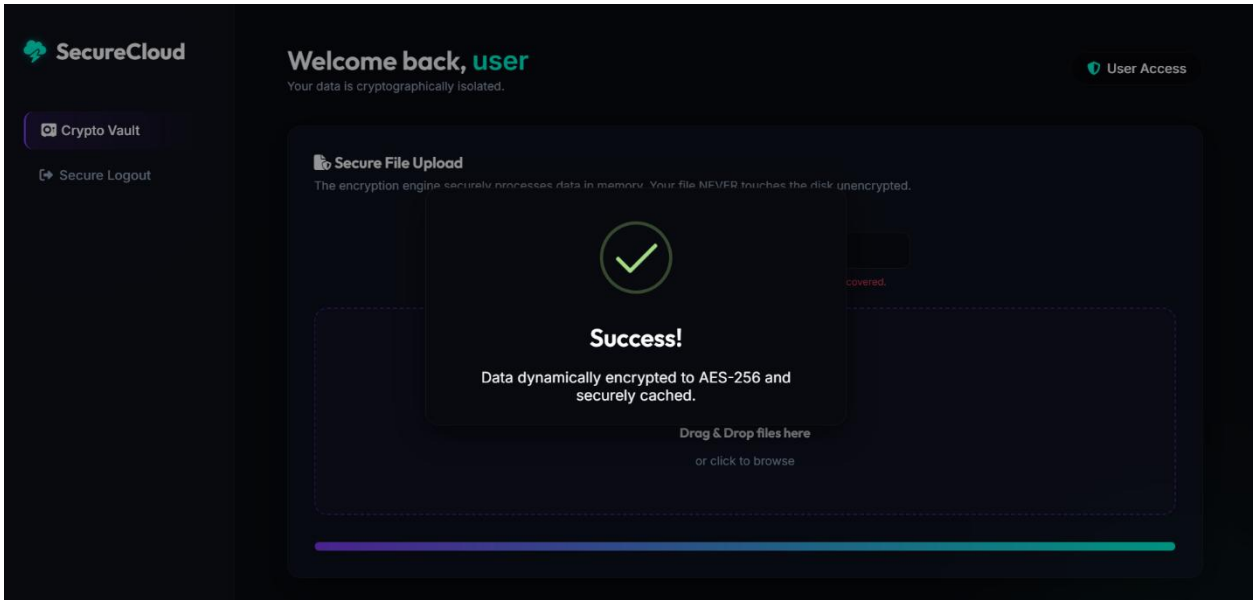
3.5.5 Secure File Upload Interface

The file upload interface is one of the most critical components of the system, as it directly interacts with the cryptographic engine. This interface allows users to select files from their local system and securely upload them to the server.

The interface includes a file selection mechanism, often enhanced with drag-and-drop functionality for ease of use. Users are required to enter an encryption password, which is used by the backend to derive a cryptographic key using PBKDF2. This password is never stored directly and is only used temporarily during the encryption process.

Once the file is submitted, the frontend sends the file data and password securely to the Flask backend using FormData. The backend then invokes the cryptographic engine to encrypt the file using AES-256-GCM before storing it. The interface provides feedback messages indicating whether the upload was successful.

This design ensures that no file is ever stored in an unencrypted form, maintaining the zero-trust principle of the system.



3.5.6 Secure File Download Interface

The file download interface allows users to retrieve their stored files in a secure manner. The interface displays a list of files associated with the user, allowing them to select a specific file for download.

Before the file is retrieved, the user is prompted to enter the decryption password. This password is used to regenerate the cryptographic key using PBKDF2. The backend then uses this key along with the stored salt and nonce values to decrypt the file using AES-GCM.

If the password is incorrect or if the file has been tampered with, the system detects the issue and prevents the download. This ensures data integrity and protects against unauthorized access.

The interface provides clear instructions and feedback messages, ensuring a smooth and secure user experience.

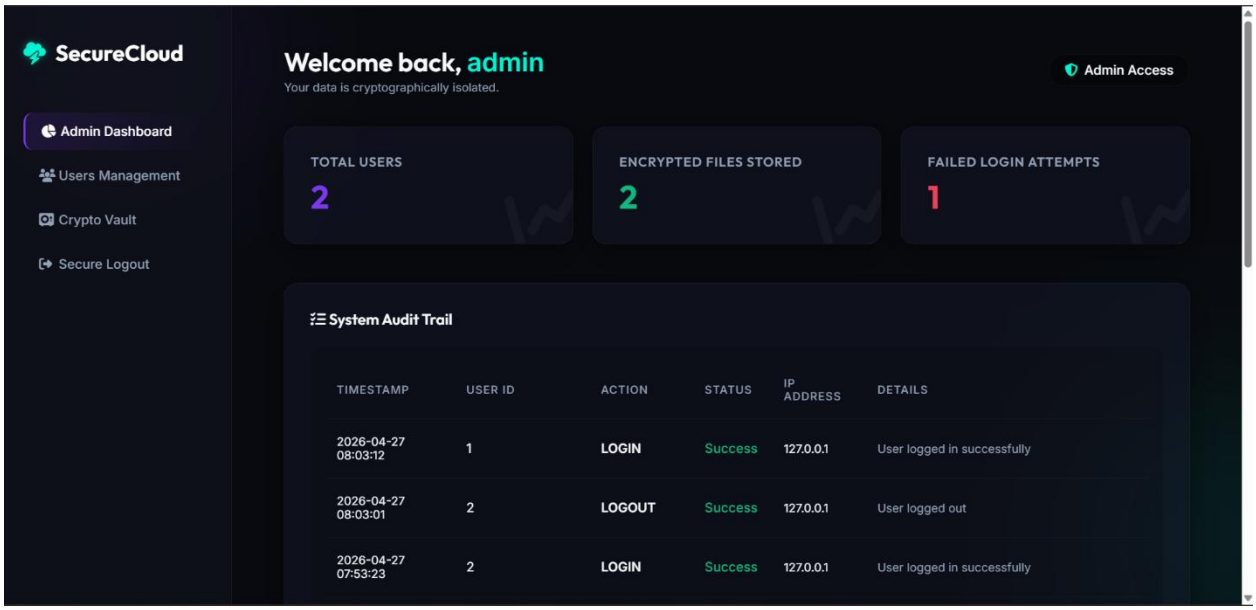
3.5.7 Admin Dashboard Interface

The admin dashboard provides advanced monitoring and control capabilities for the system administrator. This interface is essential for maintaining system security and detecting potential threats.

The dashboard displays audit logs that include details such as user actions, timestamps, IP addresses, and operation statuses. This information allows administrators to track system activity and identify suspicious behavior.

The admin interface also provides visibility into login attempts, including failed attempts, enabling quick response to potential attacks. The structured layout and filtering options make it easy to analyze large volumes of data efficiently.

This interface plays a crucial role in implementing a tamper-evident audit system, ensuring accountability and transparency in system operations.



3.5.8 User Experience and Interface Evaluation

The overall user experience of the system is designed to balance security and usability. While the system incorporates advanced cryptographic mechanisms, the GUI ensures that these complexities are hidden from the user, providing a simple and intuitive interface.

Users can perform operations such as uploading and downloading files with minimal steps, while the system handles encryption and decryption processes in the background. The use of clear instructions, structured layouts, and feedback messages enhances usability and reduces the likelihood of user errors.

The interface has been evaluated based on criteria such as ease of use, responsiveness, and security integration. The results indicate that the system provides a high-quality user experience while maintaining strong security standards.

In conclusion, the graphical user interface effectively complements the underlying hardened cloud infrastructure by providing a secure, user-friendly, and efficient platform for interaction

3.6 CUSTOMER TESTING

3.6.1 Introduction to Customer Testing

Customer testing is an essential phase in the development of any software system, as it evaluates the application from the perspective of end users rather than developers. In this project, customer testing was conducted to assess the usability, performance, and security perception of the hardened cloud infrastructure system. The primary objective of this phase was to determine how effectively the system meets user expectations while maintaining its core security features.

Unlike internal testing, which focuses on technical correctness, customer testing emphasizes real-world usage scenarios. Users interact with the system by performing typical operations such as logging in, uploading files, downloading encrypted data, and navigating through dashboards. This helps in identifying usability issues, performance bottlenecks, and areas for improvement.

In the context of this project, customer testing also plays a significant role in evaluating whether users can comfortably use a system that incorporates advanced cryptographic mechanisms without being overwhelmed by technical complexity.

3.6.2 Testing Environment and Setup

The customer testing phase was conducted in a controlled environment where users were provided access to the system through a web browser. The application was deployed locally using the Flask backend, and users interacted with it through the graphical user interface developed using HTML, CSS, and JavaScript A group of users with varying levels of technical knowledge was selected for testing.

3.6.3 Test Scenarios

During customer testing, several real-world scenarios were executed to evaluate system functionality and usability. Users were asked to perform normal operations such as successful login, file upload, and file download. In addition, they were also encouraged to test edge cases such as entering incorrect passwords, attempting multiple failed login attempts, and trying to access files without proper authorization.

Special attention was given to testing the security features of the system. Users attempted to log in with incorrect credentials multiple times to observe the behavior of the Intrusion Prevention System (IPS). The system successfully locked accounts after exceeding the allowed number of failed attempts, demonstrating its effectiveness in preventing brute-force attacks.

Users also tested the encryption and decryption workflow by uploading files with specific passwords and attempting to download them using both correct and incorrect passwords. The system correctly allowed access only when valid credentials were provided and rejected unauthorized attempts, ensuring data security.

3.6.4 Observations and User Feedback

The feedback collected from users during the testing phase was largely positive. Users found the system easy to navigate and appreciated the simplicity of the interface despite the complex security mechanisms operating in the background. The login process was described as straightforward, and users were able to quickly understand how to upload and download files.

Many users expressed satisfaction with the file upload process, particularly the clear instructions provided for entering encryption passwords. The feedback messages displayed by the system during operations such as successful uploads and failed attempts were found to be helpful and informative.

From a security perspective, users appreciated the additional protection provided by password-based encryption and account lock mechanisms. The visibility of audit logs in the admin dashboard was also highlighted as a useful feature for monitoring system activity.

However, some users suggested improvements such as enhancing the visual design of the interface, adding tooltips for better guidance, and providing mobile-friendly layouts. These suggestions indicate areas where the system can be further improved in future iterations.

3.6.5 Performance Evaluation During Testing

During the customer testing phase, the system demonstrated stable and efficient performance. File upload and encryption processes were completed within a reasonable time, depending on file size. Similarly, file download and decryption operations were executed smoothly without noticeable delays.

The system handled multiple operations without crashing or producing errors, indicating its reliability. The backend processing using Flask, combined with efficient cryptographic operations, ensured that performance remained consistent even under repeated usage.

The responsiveness of the user interface also contributed to a positive user experience. Actions performed by users were quickly reflected on the interface, providing real-time feedback and enhancing usability.

3.6.6 Challenges Faced During Testing

While the overall testing process was successful, certain challenges were observed. Some users initially found it difficult to understand the concept of encryption passwords, as it differs from regular login credentials. This highlights the need for better user guidance and instructions within the interface.

Another challenge was related to handling incorrect inputs, where users sometimes attempted to perform actions without entering required information. Although the system provided error messages, additional validation and user prompts can further improve the experience.

These challenges provide valuable insights into how the system can be refined to better meet user expectations.

3.6.7 Conclusion of Customer Testing

The customer testing phase confirms that the system is both functional and user-friendly while maintaining strong security standards. Users were able to perform all required operations successfully, and the system effectively protected data through encryption and authentication mechanisms.

The feedback received highlights the strengths of the system, including its simplicity, security features, and performance. At the same time, it provides useful suggestions for improvement, particularly in terms of user guidance and interface design.

3.7 EVALUATION

The evaluation of the developed system is carried out to measure its performance, security, usability, and reliability. This phase helps in determining how effectively the system meets its objectives of providing secure storage and controlled access to sensitive data. The results show that the system performs efficiently while maintaining strong security standards.

From a security perspective, the system provides a high level of protection by using AES-256-GCM encryption for securing files before storage. The use of PBKDF2 for key derivation ensures that encryption keys are strong and resistant to brute-force attacks. The implementation of an Intrusion Prevention System further enhances security by restricting multiple failed login attempts and preventing unauthorized access. Additionally, audit logs maintain a detailed record of all system activities, which helps in monitoring and detecting suspicious behavior.

In terms of performance, the system operates efficiently during file upload and download operations. Although encryption and decryption processes introduce some computational overhead, the system is optimized to handle these operations smoothly. The Flask backend ensures quick processing of requests and maintains effective communication between the frontend and backend components.

The usability of the system is also evaluated positively. The graphical user interface is simple, clean, and easy to navigate, allowing users to perform operations without difficulty. Features such as clear instructions, feedback messages, and structured layouts improve the overall user experience. Even users with limited technical knowledge can interact with the system comfortably.

The system also demonstrates strong reliability and stability. It handles multiple operations without errors and provides proper responses to both valid and invalid inputs. Error handling mechanisms ensure that unexpected situations are managed effectively, preventing system crashes.

4. SNAPSHOTS OF THE PROJECT

The snapshots section provides a visual representation of the system and its functionalities. It helps in understanding how the system operates in real-time and demonstrates the successful implementation of different modules.

Figure 1: Login Page

This figure shows the login interface of the system where users enter their username and password to access their account. The password field is masked to ensure confidentiality and prevent unauthorized viewing. The system validates the entered credentials through the backend, and only authenticated users are allowed to proceed. This page also integrates security features such as limited login attempts to prevent brute-force attacks.

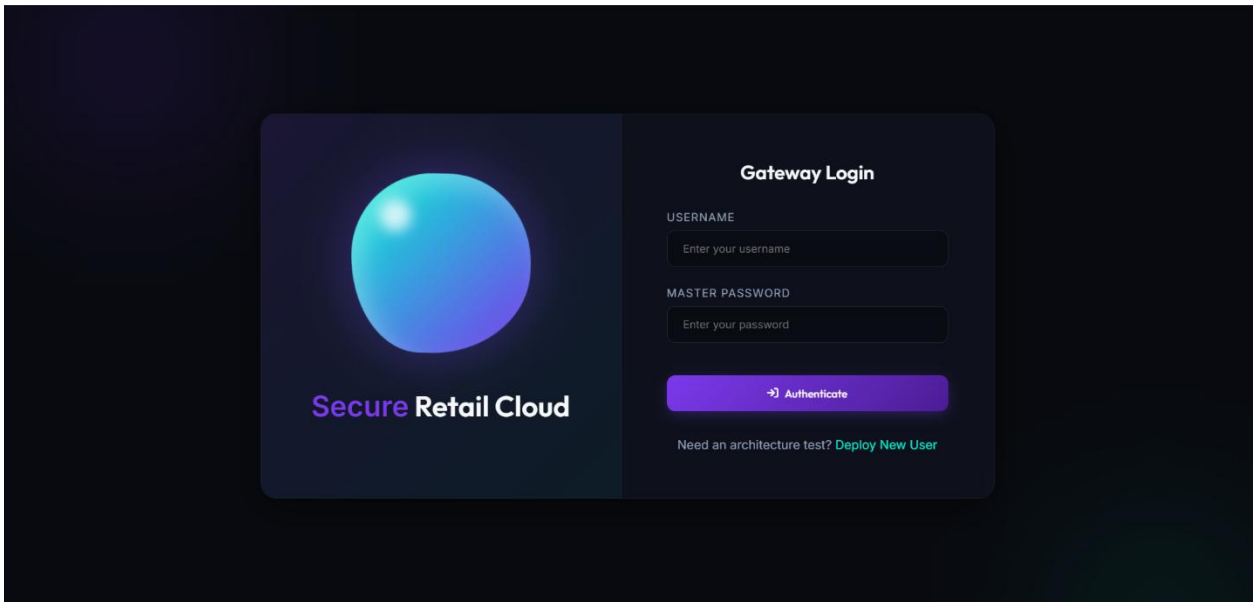


Figure 2: User Dashboard

This figure represents the user dashboard, which acts as the main interface after successful login. It provides access to all major functionalities of the system, including file upload and file download. The dashboard is designed to be simple and easy to navigate, allowing users to perform operations efficiently. It may also display user-related information and system status.

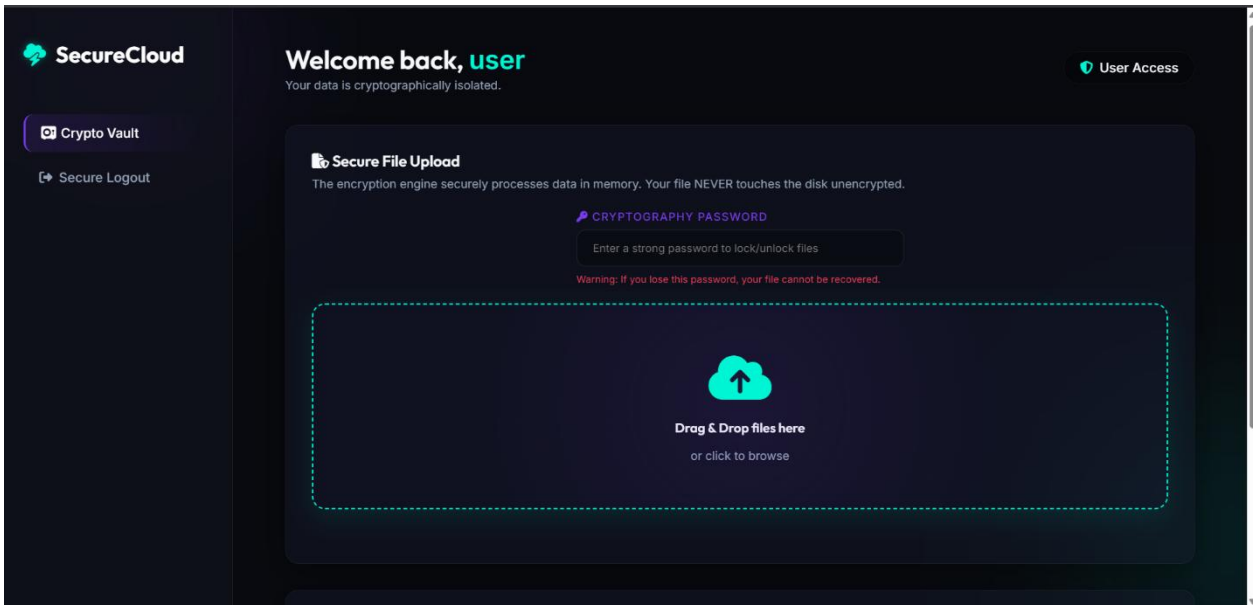


Figure 3: File Upload Interface

This figure shows the file upload interface where users can securely upload files to the system. The user selects a file from their local system and provides an encryption password. This password is used to generate a secure encryption key using PBKDF2. The selected file is then encrypted using AES-256-GCM before being stored in the system, ensuring that the data remains protected.

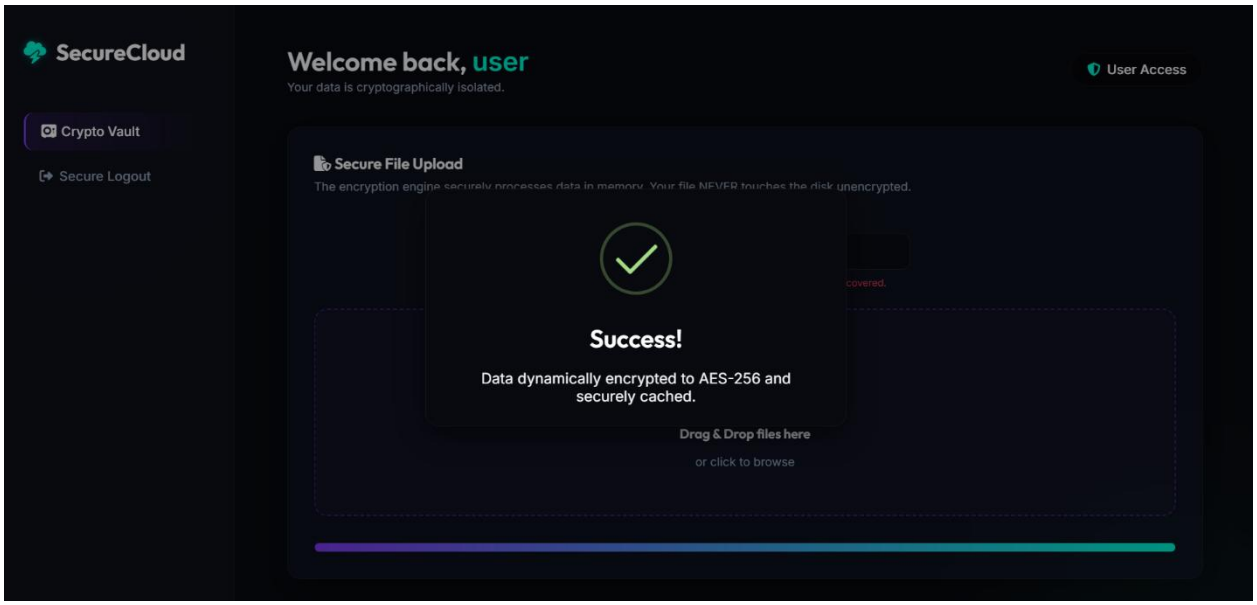


Figure 4: File Download Interface

This figure illustrates the file download process. Users can select a previously uploaded file and are required to enter the correct decryption password. The system uses this password to regenerate the encryption key and decrypt the file securely. If the correct password is provided, the file is successfully downloaded; otherwise, access is denied to maintain data security.

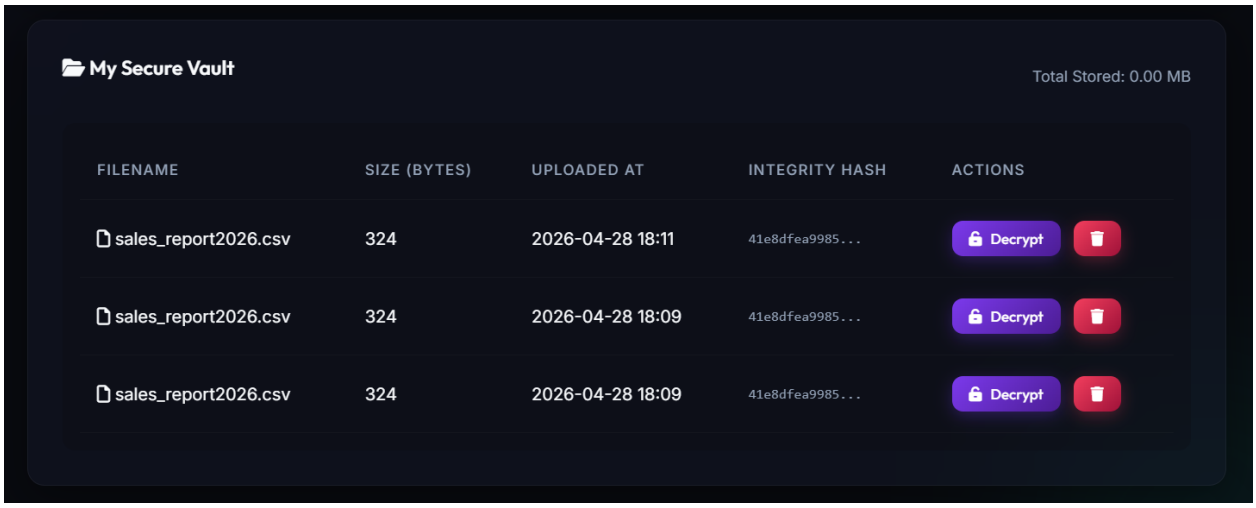


Figure 5: Admin Dashboard

This figure shows the admin dashboard, which provides monitoring and control features for the system administrator. It allows viewing of user activities, login attempts, and file operations. The admin can use this interface to track system usage and detect any suspicious behavior, thereby improving system security and management.

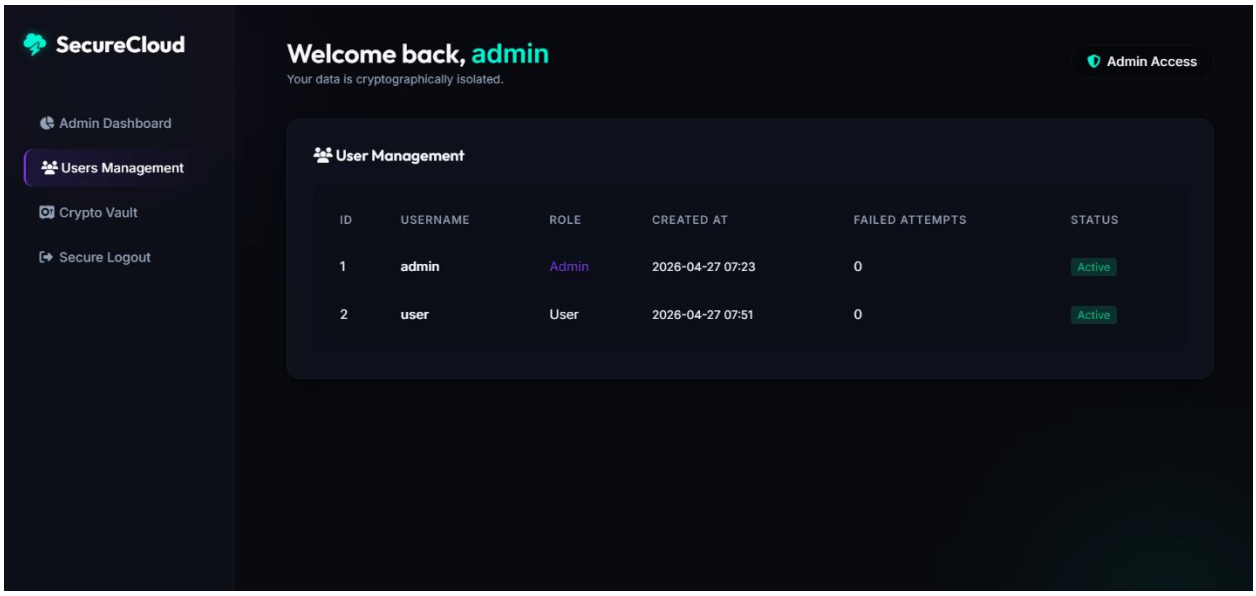
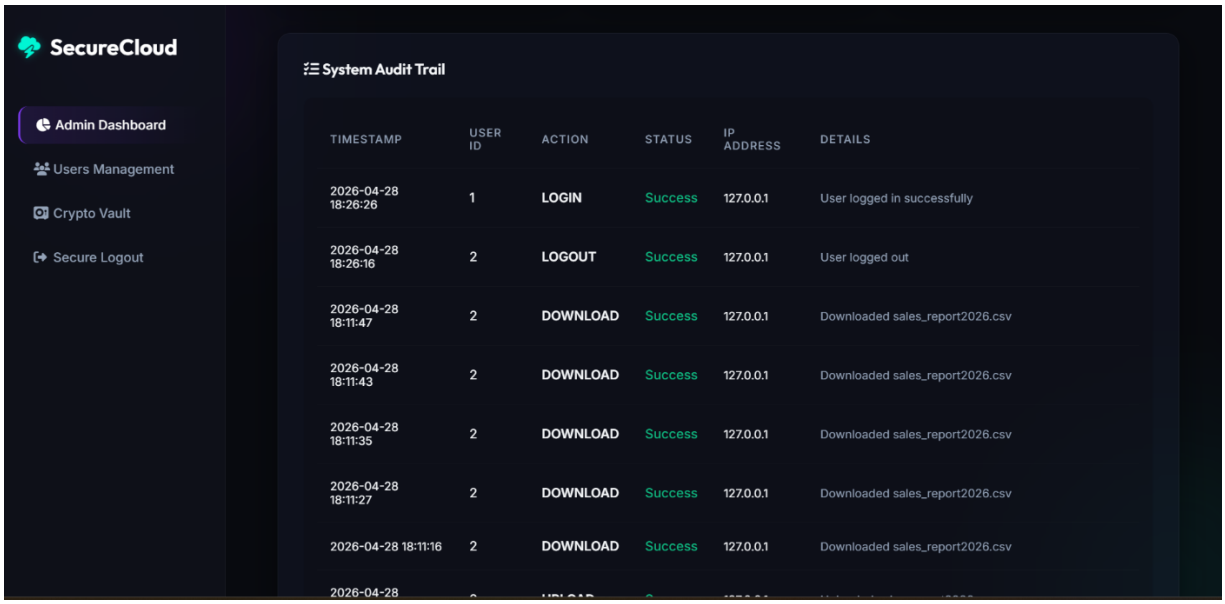


Figure 6: Audit Logs

This figure displays the audit logs maintained by the system. The logs include detailed records of user actions such as login attempts, file uploads, and downloads, along with timestamps and IP addresses. This feature helps in tracking system activity and provides transparency, making it easier to identify unauthorized or suspicious actions.



5. CONCLUSION

The project "Hardened Cloud Infrastructure for Secure Retail Data Storage" successfully achieves its objective of providing a secure and efficient method for storing sensitive data. The system ensures data protection by using AES-256-GCM encryption, which encrypts files before storage, making them inaccessible to unauthorized users. The use of PBKDF2 key derivation further strengthens security by generating strong encryption keys from user-provided passwords.

In addition to encryption, the system incorporates an Intrusion Prevention System that prevents unauthorized access by limiting repeated failed login attempts. The audit logging feature records user activities such as login attempts, file uploads, and downloads, which improves monitoring and accountability.

The graphical user interface is designed to be simple and user-friendly, allowing users to interact with the system without requiring technical knowledge. The system performs efficiently and reliably, ensuring smooth file operations while maintaining strong security. Overall, the project provides a practical and effective solution for secure cloud-based data storage.

6. FURTHER DEVELOPMENT OR RESEARCH

The current system provides a strong foundation for secure data storage; however, it can be further enhanced in several ways. One of the major improvements is integrating the system with cloud platforms such as AWS or Microsoft Azure. This will allow the system to scale efficiently and support large volumes of data.

Another important enhancement is the implementation of multi-factor authentication, which will add an extra layer of security during login. This will significantly reduce the risk of unauthorized access. The system can also be improved by implementing password strength validation to ensure users choose secure passwords.

Future research can focus on incorporating artificial intelligence techniques to detect suspicious activities and prevent cyber threats. Additionally, developing a mobile application version of the system can improve accessibility and user convenience. Enhancing the graphical user interface will further improve the overall user experience.

7. REFERENCES

The development of this project is based on various standard documentation, research papers, and technical resources related to web development, cryptography, and cloud security. These references provided both theoretical knowledge and practical guidance for implementing a secure system.

Flask Official Documentation was used for understanding backend development, routing, and request handling in the web application. The Python Cryptography Library documentation provided detailed guidance for implementing AES-256-GCM encryption and PBKDF2 key derivation techniques.

Research papers on Advanced Encryption Standard (AES) helped in understanding encryption mechanisms and their importance in securing sensitive data. Documentation and articles related to PBKDF2 explained how password-based key derivation improves resistance against brute-force attacks.

Additional resources such as cybersecurity articles and IEEE research papers on cloud security and data protection were also referred to during the development process. These sources helped in designing a system that follows modern security practices and addresses real-world challenges.

8. APPENDIX

The appendix section provides supplementary information that supports the main content of the project report. It includes additional technical details that are useful for understanding the system implementation in depth but are not included in the main sections to maintain clarity.

This section contains important elements such as sample source code used for file encryption and decryption, which demonstrates how AES-256-GCM and PBKDF2 are implemented in the system. It may also include database schema details showing the structure of tables such as user records, file records, and audit logs.

Further details such as API endpoints used for communication between the frontend and Flask backend can also be included. These endpoints describe how data is transmitted securely during operations like login, file upload, and file download.

Additional diagrams, configuration settings, and testing data used during development can also be placed in this section. The appendix acts as a reference for readers who wish to explore the technical aspects of the system in more detail and provides a complete understanding of how the system is built and functions.